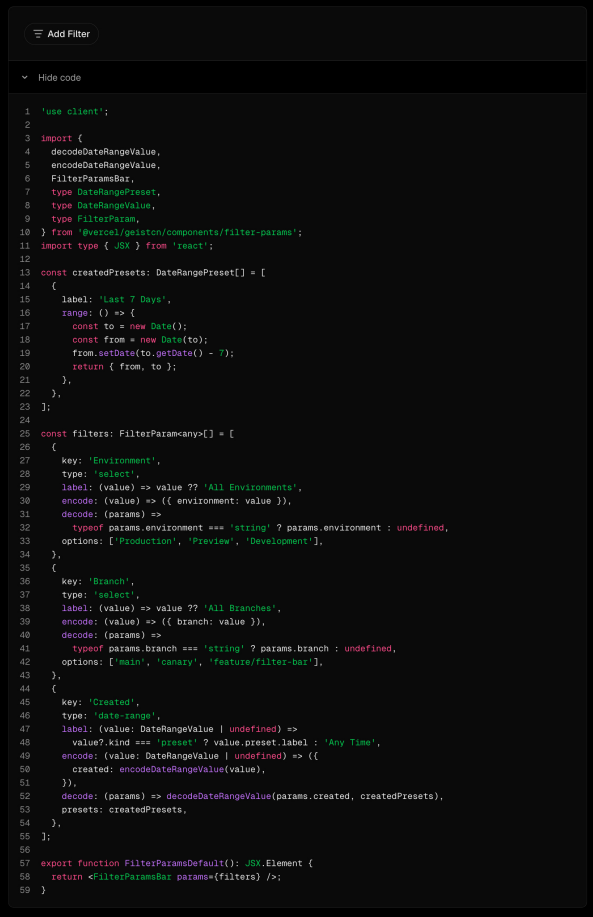
Command Menu
Context Card
Context Menu
Copy Button
Description
Destructive Action Modal
Destructive Confirmation Input
Dots Menu
Drawer
Empty State
Entity
Error
Error Card
Feedback
Fieldset
File Tree
Filter Params
Gauge
Grid
Input
JSON View
Keyboard Input
Label
Load More Button
Loading Dots
Menu
MiddleTruncate
Modal

Filter Params

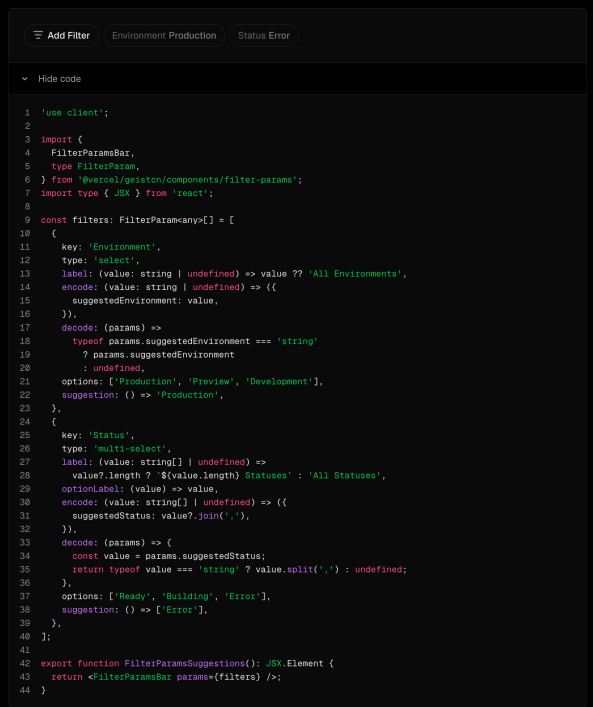
A URL-backed filter bar with searchable filter selection.

Default



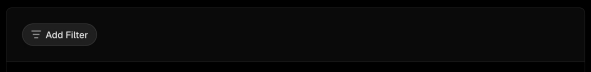
Suggestions

Provide a `suggestion` for filters that have a useful starting value. Suggestions remain optional until users select them.



AI

Pass an `ai` callback to offer natural-language filter suggestions. This example simulates a request and always applies the Production environment filter.



Hide code

```tsx
'use client';

import {
  FilterParamsBar,
  useSearchParamState,
  type FilterParam,
} from '@vercel/geistcn/components/filter-params';
import { useCallback, useRef, useState, type JSX } from 'react';

const filters: FilterParam<string>[] = [
  {
    key: 'Environment',
    type: 'select',
    label: (value) => value ?? 'All Environments',
    encode: (value) => ({ aiEnvironment: value }),
    decode: (params) =>
      typeof params.aiEnvironment === 'string'
        ? params.aiEnvironment
        : undefined,
    options: ['Production', 'Preview', 'Development'],
  },
];

export function FilterParamsAi(): JSX.Element {
  const [searchParams, setSearchParams] = useSearchParamState();
  const [isThinking, setIsThinking] = useState(false);
  const requestId = useRef(0);

  const onAbort = useCallback(() => {
    requestId.current += 1;
    setIsThinking(false);
  }, []);

  const onQuery = useCallback(async (): Promise<number | void> => {
    const id = ++requestId.current;
    setIsThinking(true);
    await new Promise((resolve) => setTimeout(resolve, 3000));

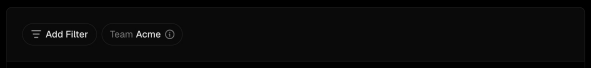
    if (id !== requestId.current) return;

    const nextSearchParams = new URLSearchParams(searchParams);
    nextSearchParams.set('aiEnvironment', 'Production');
    setSearchParams(nextSearchParams);
    setIsThinking(false);
    return 1;
  }, [searchParams, setSearchParams]);

  return (
    <FilterParamsBar ai={{ isThinking, onAbort, onQuery }} params={filters} />
  );
}
```

Readonly filters

Use a readonly filter for page context that should be visible but cannot be changed or removed. It still uses `encode` and `decode`, so its value is available through `useFilterParam` and can be restored optimistically on refresh. `onLoad` supplies the authoritative value when the bar renders. Add a `note` to explain the context in a hover card.



Hide code

```tsx
'use client';

import {
  FilterParamsBar,
  type FilterParam,
} from '@vercel/geistcn/components/filter-params';
import type { JSX } from 'react';

const filters: FilterParam<any>[] = [
  {
    key: 'Team',
    type: 'readonly',
    label: (value: string | undefined) => value ?? '',
    onLoad: async () => 'Acme',
    encode: (value: string | undefined) => ({ team: value }),
    decode: (params) =>
      typeof params.team === 'string' ? params.team : undefined,
    note: 'The current team is set by the page context and cannot be changed.',
  },
  {
    key: 'Environment',
    type: 'select',
    label: (value: string | undefined) => value ?? 'All Environments',
    encode: (value: string | undefined) => ({ environment: value }),
    decode: (params) =>
      typeof params.environment === 'string' ? params.environment : undefined,
    options: ['Production', 'Preview', 'Development'],
  },
];

export function FilterParamsReadonly(): JSX.Element {
  return <FilterParamsBar params={filters} />;
}
```

Best Practices

When to use

- Use `FilterParamsBar` for list and search pages where the active constraints should be visible, linkable, and preserved on refresh.
- Keep filter names short and use option labels that explain the selected value without requiring users to reopen the menu.

Behavior

- Encode only the URL parameters owned by each filter so removing a pill does not clear unrelated page state.
- Provide suggestions only when they are useful defaults, not as replacements for filters users have explicitly selected.

Accessibility

- Make filter labels and option labels specific enough to identify the constraint and selected value when announced by a screen reader.

by a screen reader.